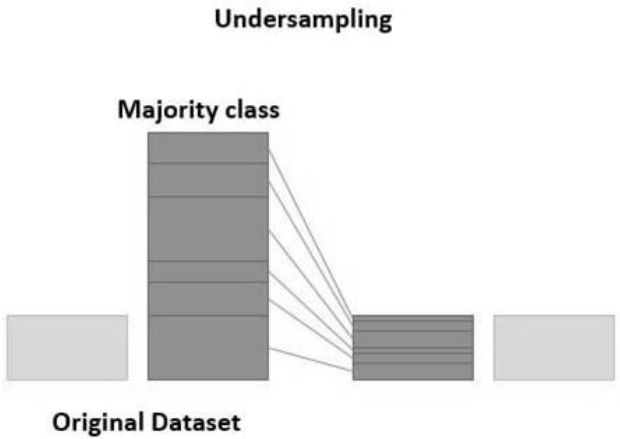
# Balansarea claselor (pt. clasificare)

- Dacă o clasă este dominantă (de exemplu mai mult de 90% din observații), devin greu de construit un model de clasificare
- În mod ideal, când antrenăm un model de clasificare cu n clase, datele de antrenament ar trebui să conțină un număr similar de observații pentru fiecare clasă
- În realitate nu se întâmplă așa. Uneori este benefică balansarea claselor
  - Prin supraeșantionare (oversampling), sau
  - Prin subeșantionare (undersampling)

# Balansarea claselor prin subeșantionare (undersampling)

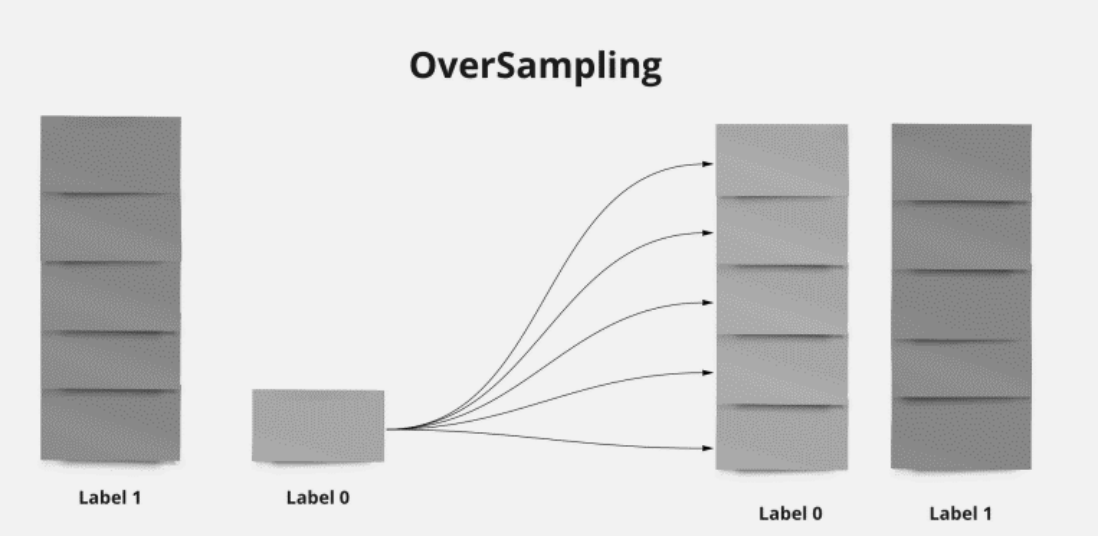
- Cea mai simplă strategie – din clasa mare se eșantionează de mărimea clasei mai mici
- Funcționează bine pentru Big Data




- Pentru date puține (ca în psihologie) nu este benefică
  - De exemplu dacă avem 100 de observații cu clasele 0 și 1 și 5 sunt cu clasa 0 restul clasa 1, subeșantionarea ne dă un set cu înregistrări – 5 cu clasa 1 și 5 cu clasa 0. Cam greu de făcut statistică cu 10 numere –contraexemplu:

# Balansarea claselor prin supraeșantionare (oversampling)

- Strategia pornește cu clasa mai mică
- Se generează sintetic observații asemănătoare celor din clasa mică până când mărimea eșantionului este similară cu cea a clasei mari



# Balansarea claselor – metode Python

- Resurse Python
  - https://imbalanced-learn.org/stable/
  - https://learn-scikit.oneoffcoder.com/imbalanced-learn.html

This is the full API documentation of the `imbalanced-learn` toolbox.

Under-sampling methods
- Prototype generation
  - ClusterCentroids
- Prototype selection
  - CondensedNearestNeighbour
  - EditedNearestNeighbours
  - RepeatedEditedNearestNeighbours
  - AllKNN
  - InstanceHardnessThreshold
  - NearMiss
  - NeighbourhoodCleaningRule
  - OneSidedSelection
  - RandomUnderSampler
  - TomekLinks

Over-sampling methods
- Basic over-sampling
  - RandomOverSampler
- SMOTE algorithms
  - SMOTE
  - SMOTENC
  - SMOTEN
  - ADASYN
  - BorderlineSMOTE
  - KMeansSMOTE
  - SVMSMOTE

# Atenție: balansați doar datele de antrenament

- Împărțiți întâi train/test (cu stratify=y, ca testul să păstreze proporția reală a claselor)
- Subeșantionarea sau supraeșantionarea se aplică numai pe train
- Testul rămâne cu distribuția reală: acolo se măsoară modelul
- Dacă balansați înainte de împărțire, observații sintetice sau copii ale aceluiași client ajung și în test: scorul pare mai bun decât este
- Alternativă simplă: class_weight="balanced" în model, fără a modifica datele